

MASTER 2 — SIB · MONÉTIQUE ET MOYENS DE PAIEMENT

DevOps et Automatisation pour les Systèmes d'Information Bancaire

TRAVAUX PRATIQUES

Chaîne DevOps de bout en bout : Git · Docker · PostgreSQL · Nginx · CI/CD

Projet de groupe — squelette imposé payment-api

Énoncé consolidé à l'attention des étudiants

Modalité : travail en groupe · Livraison sur dépôt GitHub

Sommaire

Sommaire.....	2
1. Contexte et objectifs pédagogiques	3
Compétences visées	3
2. Architecture imposée (socle commun).....	3
2.1 Pile technique	3
2.2 Schéma de flux.....	3
3. Répartition et sujets de TP.....	4
4. Chaîne DevOps attendue	5
4.1 Versionnement et collaboration (Git / GitHub)	5
4.2 Conteneurisation (Docker / Docker Compose).....	5
4.3 Intégration continue (GitHub Actions)	5
5. Exigences de sécurité.....	5
6. Livrables attendus.....	6
7. Grille de notation	6

1. Contexte et objectifs pédagogiques

Ce dossier regroupe les cinq sujets de travaux pratiques du module. Chaque groupe se voit attribuer un sujet unique et doit livrer une application bancaire complète, non pas au sens de sa richesse fonctionnelle, mais au sens de sa chaîne de production : versionnement, conteneurisation, persistance des données et automatisation des tests.

La logique métier de chaque sujet est volontairement simple (un formulaire de saisie exposé par une API).

L'objet réellement évalué n'est pas l'application, mais la maîtrise de la chaîne DevOps qui la produit et la déploie.

Compétences visées

- Versionner un projet avec Git et collaborer via un dépôt GitHub distant (modèle branche → pull request → fusion).
- Concevoir et exposer une API REST persistée dans une base de données PostgreSQL.
- Traduire des règles de validation métier bancaires en cas de tests automatisés.
- Conteneuriser une application multi-services avec Docker et Docker Compose, derrière un reverse proxy Nginx.
- Mettre en place un pipeline d'intégration continue avec GitHub Actions.
- Appliquer les bonnes pratiques élémentaires de gestion des secrets, en cohérence avec les exigences PCI-DSS et ISO 27001.

2. Architecture imposée (socle commun)

Pour garantir l'équité de notation entre les groupes, l'architecture est **identique pour tous les sujets**. Seuls le modèle de données et les règles de validation changent d'un sujet à l'autre.

2.1 Pile technique

Composant	Rôle et exigence
API REST	Application Python Flask exposant des endpoints CRUD. Point de départ imposé : le squelette <code>payment-api</code> fourni.
Base de données	PostgreSQL en conteneur. La persistance doit être réelle : aucune donnée stockée en mémoire ou dans un fichier plat.
Reverse proxy	Nginx en frontal, point d'entrée unique. L'API n'est jamais exposée directement : tout le trafic passe par Nginx.
Tests	pytest. Chaque règle de validation métier du sujet doit être couverte par au moins un test.
Orchestration	<code>docker-compose.yml</code> démarrant les trois services (api, db, nginx) en une seule commande.
Intégration continue	GitHub Actions : lint, tests et build de l'image, déclenchés sur push et pull_request.

2.2 Schéma de flux

Client (formulaire / curl) → Nginx (reverse proxy, :80) → API Flask (:5000) → PostgreSQL (:5432)

Un schéma d'architecture reprenant ce flux doit figurer dans le **README** du dépôt.

3. Répartition et sujets de TP

Chaque groupe traite un et un seul sujet. Les champs listés définissent le modèle de données ; les règles de validation définissent le comportement attendu de l'API et, par conséquent, les cas de tests pytest à écrire.

TP1 — Application de gestion de virement simple

Champs du formulaire : compte à débiter, compte à créditer, montant, motif — boutons Valider et Réinitialiser.

Règles de validation attendues :

- Les deux comptes respectent un format RIB/IBAN valide (contrôle de longueur et de structure).
- Le compte à débiter est différent du compte à créditer.
- Le montant est un décimal strictement positif, à deux chiffres après la virgule.
- La devise appartient à une liste fermée (XAF, EUR, USD) ; défaut : XAF.
- Le motif est obligatoire et limité à 140 caractères.

TP2 — Enregistrement des informations de prospects

Champs du formulaire : nom, prénom, date de naissance, lieu de naissance, profession, secteur d'activité, téléphone, e-mail.

Règles de validation attendues :

- Nom et prénom obligatoires, alphabétiques, au moins 2 caractères.
- Date de naissance non future et cohérente avec la majorité (18 ans révolus).
- E-mail conforme (validation par expression régulière) et unique.
- Téléphone au format Gabon (+241 XX XX XX XX) ou format international valide.
- Secteur d'activité choisi dans une liste fermée (référentiel fourni par le groupe).

TP3 — Application de paiement marchand

Champs du formulaire : identifiant de la caisse, nom du client, téléphone du client, adresse e-mail, montant.

Règles de validation attendues :

- L'identifiant de caisse existe dans un référentiel de caisses (liste fermée).
- Le montant est strictement positif.
- Au moins un canal de contact valide est fourni (téléphone OU e-mail).
- Si fourni, l'e-mail est conforme et le téléphone respecte le format attendu.
- Une référence de transaction unique est générée à l'enregistrement.

TP4 — Souscription d'un produit par un client

Champs du formulaire : code client, nom du client, produit souscrit, date de souscription.

Règles de validation attendues :

- Le code client respecte un format défini (ex. 8 chiffres) et est obligatoire.
- Le produit souscrit appartient à un catalogue fermé (liste de produits).
- La date de souscription n'est pas dans le futur.
- Un même client ne peut pas souscrire deux fois le même produit (unicité code client + produit).

TP5 — Mise en place d'un crédit

Champs du formulaire : code client, type de crédit, durée du crédit, montant, garantie (oui/non).

Règles de validation attendues :

- Le code client respecte un format défini et est obligatoire.
- Le type de crédit appartient à une liste fermée (consommation, immobilier, trésorerie...).
- La durée est un entier strictement positif, dans une plage bornée (ex. 1 à 360 mois).
- Le montant est strictement positif.
- Règle conditionnelle : pour un crédit immobilier, la garantie est obligatoire (oui).
- Bonus : calcul et restitution de la mensualité (montant, durée, taux).

4. Chaîne DevOps attendue

4.1 Versionnement et collaboration (Git / GitHub)

1. Installer Git et configurer l'identité de chaque membre (nom, e-mail).
2. Cloner le squelette `payment-api` et créer un dépôt GitHub distant pour le groupe.
3. Travailler selon le modèle **branche de fonctionnalité → pull request → fusion** : aucun commit direct sur `main`.
4. Au moins **3 commits par membre**, avec des messages clairs. La contribution individuelle sera vérifiée via `git log --author`.
5. Relier le projet local au dépôt distant et pousser l'ensemble du travail sur GitHub.

4.2 Conteneurisation (Docker / Docker Compose)

- Écrire le `Dockerfile` de l'API.
- Écrire le `docker-compose.yml` orchestrant trois services : `api`, `db` (PostgreSQL) et `nginx`.
- Configurer Nginx en reverse proxy devant l'API ; l'API n'est jamais publiée directement à l'extérieur.
- Démarrer l'ensemble avec `docker compose up` et vérifier que l'API répond bien à **travers Nginx**.
- Bonus : lancer l'API en **2 réplicas** derrière Nginx pour illustrer la répartition de charge, ou activer une terminaison TLS.

4.3 Intégration continue (GitHub Actions)

Mettre en place un pipeline déclenché sur `push` et `pull_request`, enchaînant au minimum les étapes suivantes :

- Lint / formatage (`flake8` et/ou `black --check`).
- Tests unitaires (`pytest`) couvrant chaque règle de validation du sujet.
- Build de l'image Docker de l'API.
- Bonus : analyse de sécurité (`bandit` sur le code, ou `Trivy` sur l'image).

5. Exigences de sécurité

Ces applications manipulent des données bancaires. Les règles suivantes sont minimales et non négociables, en cohérence avec les principes PCI-DSS et ISO 27001 abordés en cours.

- Aucun secret en clair dans le code : mot de passe de base de données, clés et identifiants passent par variables d'environnement / fichier `.env`.

- Le fichier `.env` est exclu du dépôt via `.gitignore` ; un `.env.example` sans valeur sensible est fourni à la place.
- Dans la CI, les secrets éventuels sont stockés dans `GitHub Secrets`, jamais en dur dans le workflow.
- Bonus : intégration d'un scan de vulnérabilités (image ou code) au pipeline.

6. Livrables attendus

- Lien du dépôt GitHub du groupe (public, ou accès accordé à l'enseignant).
- Un `README` comprenant : description du sujet, schéma d'architecture, instructions de lancement en une commande, et liste des règles de validation implémentées.
- Le `docker-compose.yml` fonctionnel démarrant les trois services.
- Capture d'écran du pipeline GitHub Actions au vert (statut de la dernière exécution).
- La liste des membres du groupe et leur répartition des tâches.

7. Grille de notation

Notation sur 20 points, avec 2 points de bonus possibles. La grille sera appliquée à l'identique pour tous les groupes.

Critère	Attendu	Barème
Git & collaboration	Branches, pull requests, commits répartis par membre, historique propre.	3
API REST & données	Endpoints CRUD fonctionnels, modèle persisté dans PostgreSQL.	3
Validation métier & tests	Règles du sujet implémentées et couvertes par des tests pytest.	4
Conteneurisation	Dockerfile, docker-compose (api + db + nginx), API accessible via Nginx.	4
Pipeline CI	GitHub Actions : lint + tests + build, sur push et pull request.	3
Sécurité	Secrets hors du code, <code>.env</code> ignoré, <code>.env.example</code> fourni.	2
Documentation	README complet avec schéma d'architecture et instructions de lancement.	1
Total		20

Rappel : un rendu qui « tourne en local » mais dont le pipeline est en échec, ou dont les secrets sont versionnés en clair, sera pénalisé même si l'application fonctionne. La chaîne d'automatisation est l'objet du module.